WorksheetDownload.com

10 CONCEPTUAL Q&A MOBILE · PRACTICE WORKSHEET

CONCEPTUAL SERIES
Swift

Name: ______ Cohort/Batch: ______ Date: ______ Score: ______

SWIFT PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Mobile

TOTAL: 10 QUESTIONS

Q1 What is Swift and what UI problem does it address? *Threat Model*

Q2 How do components communicate in Swift? *Architecture*

Q3 How does Swift manage state for complex applications? *Configuration*

Q4 What rendering strategies does Swift support? *Hardening*

Q5 How does Swift handle client-side routing? *SSO / IdP*

Q6 How do you fetch data and handle loading/error states in Swift? *Observability*

Q7 What performance techniques apply to Swift applications? *Lifecycle*

Q8 How do you handle forms and validation in Swift? *Compliance*

Q9 How do you test Swift components? *Testing*

Q10 What accessibility (a11y) practices are essential in Swift? *Tuning*

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Swift is a component-based library or

Mitigation

Swift is a component-based library or framework for building interactive user interfaces. It provides a reactive programming model where UI updates automatically when underlying state changes, reducing imperative DOM manipulation.

A2 Parent-to-child communication uses props

Primitives

Parent-to-child communication uses props (one-way data flow). Child-to-parent uses events or callbacks. Siblings share state via a common parent or a global store (Redux, Pinia, Signals). Avoid prop-drilling by using context or state management.

A3 Local component state handles UI-only concerns.

Hardening

Local component state handles UI-only concerns. Shared state (user auth, cart) lives in a global store with a predictable update pattern (actions, reducers, or mutations). Server state (API data) is managed by libraries like React Query or SWR.

A4 Swift supports CSR (browser renders), SSR

Gotchas

Swift supports CSR (browser renders), SSR (server renders HTML on each request), SSG (pages generated at build time), and ISR (pages regenerated on a schedule or on-demand). Choose based on SEO requirements and data freshness needs.

A5 Swift's router (built-in or a library

Federation

Swift's router (built-in or a library like React Router) maps URL paths to components without page reloads. Dynamic segments (/users/:id) are parsed and passed as params. Guards or middleware can protect routes requiring authentication.

A6 Use a data fetching hook or

Observability

Use a data fetching hook or library that returns { data, isLoading, error }. Show a skeleton or spinner while loading, an error message on failure, and the data on success. Avoid fetching in render to prevent waterfalls.

A7 Code splitting loads only the code

Automation

Code splitting loads only the code needed for the current route. Memoization (useMemo, React.memo) prevents unnecessary re-renders. Virtualization renders only visible list items. Images use lazy loading and next-gen formats (WebP, AVIF).

A8 Swift manages form state with controlled

Governance

Swift manages form state with controlled inputs or a form library (React Hook Form, VeeValidate). Validation runs on blur or submit, displaying field-level error messages. Schema libraries (Zod, Yup) define validation rules declaratively.

A9 Unit tests check individual component

Assessment

Unit tests check individual component rendering and interactions using a component testing library. Integration tests verify multi-component flows. End-to-end tests (Playwright, Cypress) simulate real user journeys in a browser.

A10 Use semantic HTML (button, nav, main)

Optimization

Use semantic HTML (button, nav, main) instead of divs with click handlers. Ensure keyboard navigability (tab order, focus management). Add ARIA roles and labels for dynamic content. Test with a screen reader and run axe or Lighthouse audits.